Name – Ahmed Hussain Shaikh

Documentation of Frontend Deployment Architecture and Pipeline Workflow

AWS Resources Used Here:

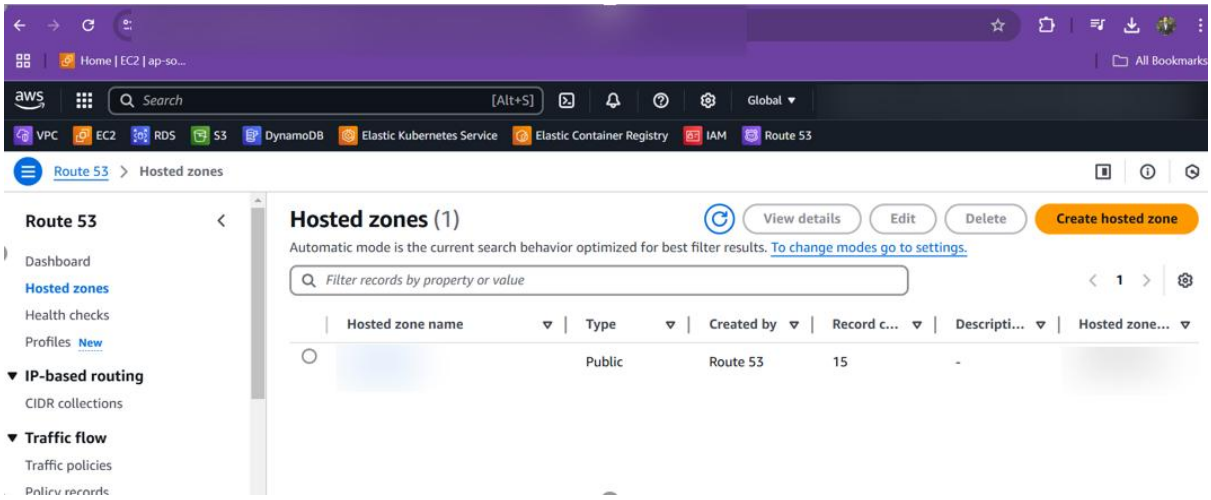
- Route 53
- Cloudfront
- Application Load Balancer
- ACM
- S3 Bucket
- Jenkins

1. Domain and Subdomain Setup

Primary Domain (Route 53)

Domain Name: astra.in

Hosted in: AWS Route 53

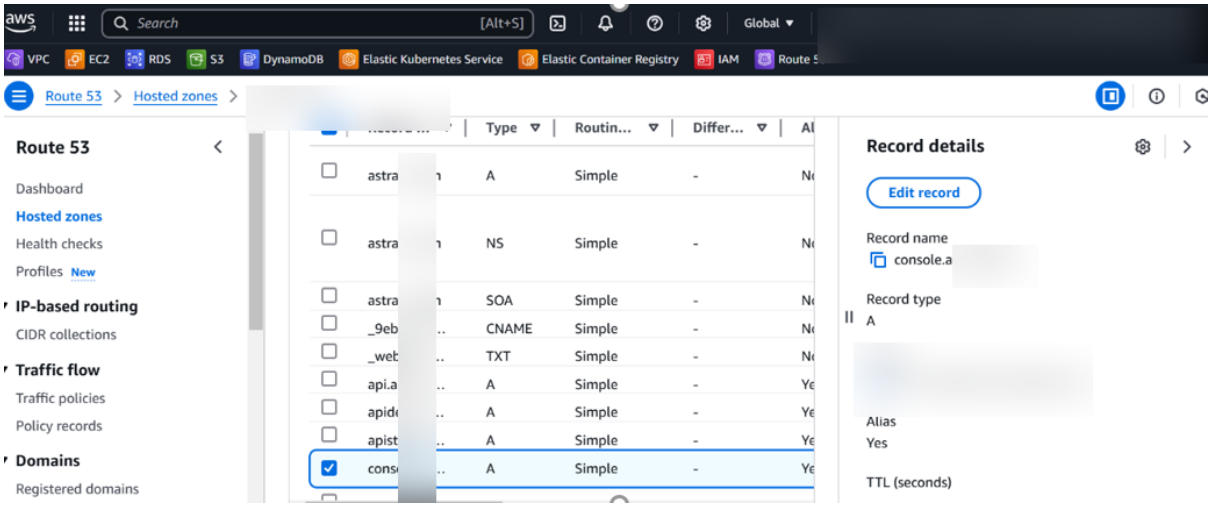


Acts as the primary DNS for our infrastructure.

Subdomain for Production EO Application Frontend (console.astra.in)

Record Type: A Record

Value: Points to a CloudFront distribution.



So whenever any user sends the Requests to console.astra.in the requests are routed to CloudFront, which caches and serves content globally.

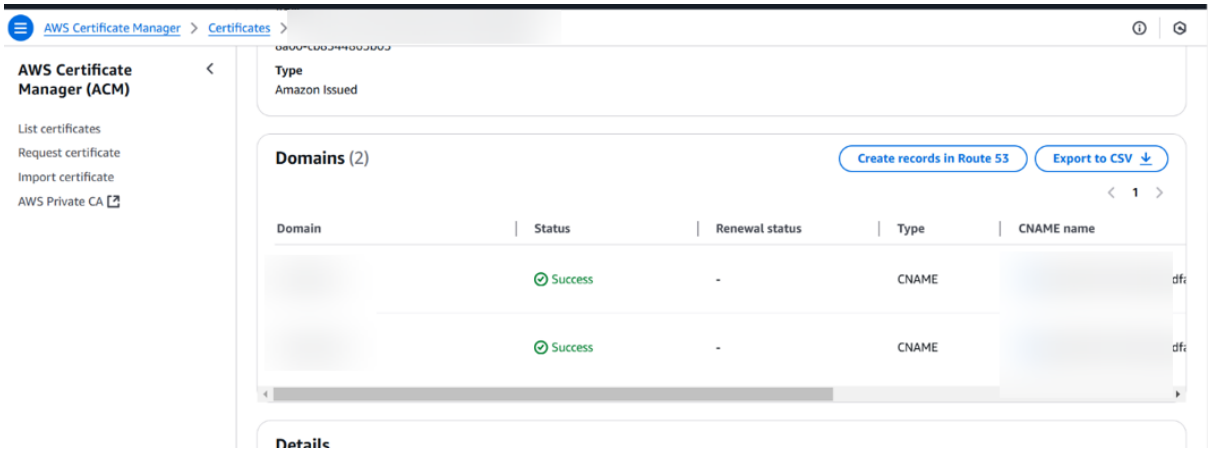
2. CloudFront Configuration

Alternate Domain Name

CloudFront recognizes console.astra.in as a valid alias for the distribution as we have already added the domain name in cloudfront distribution.

SSL/TLS Security

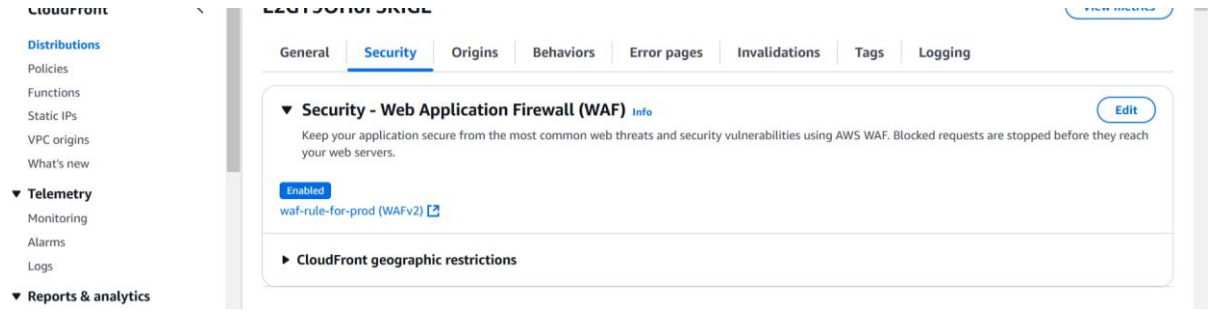
Certificate: An ACM (AWS Certificate Manager) certificate ensures encrypted communication (HTTPS).



3. WAF Rules

A Web Application Firewall (WAF) rule is applied for Geo-based IP restrictions here.

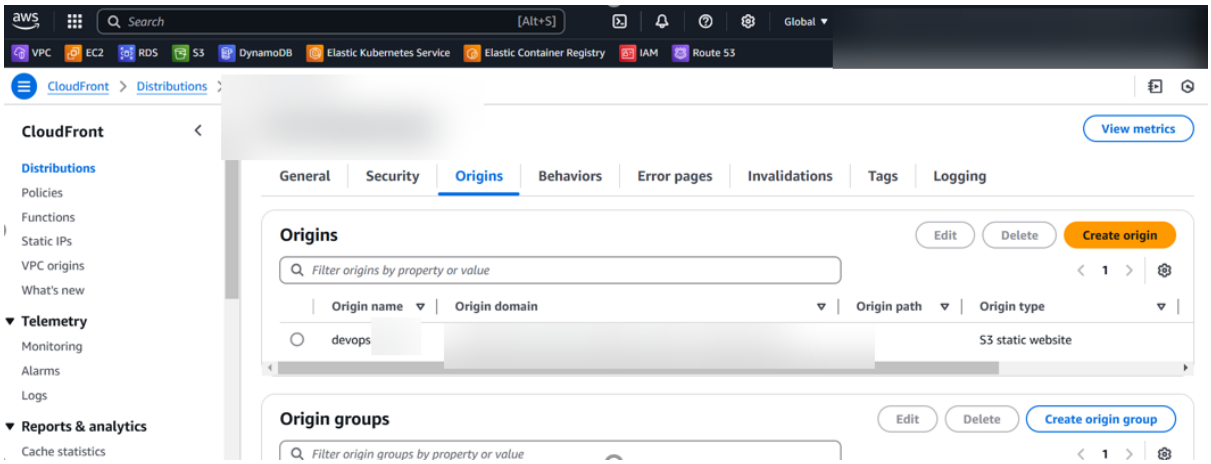
Restriction: Only requests originating from India are allowed.



4. CloudFront Origin

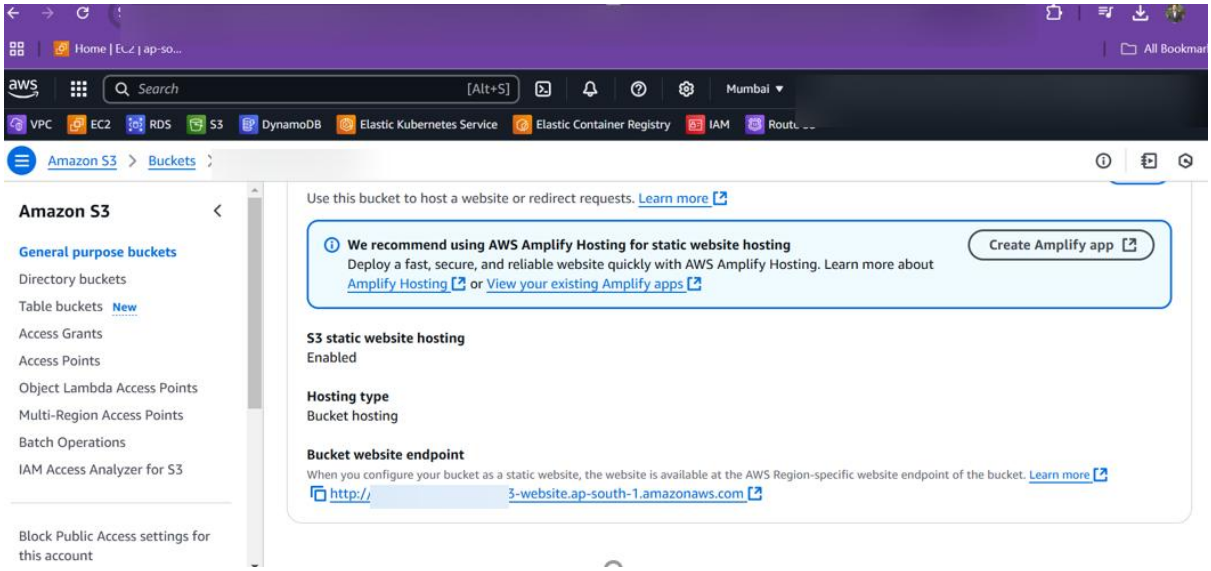
Origin: Points to the S3 static web hosting endpoint.

The frontend (UI) files are hosted in the S3 bucket, and CloudFront serves these files efficiently with global caching.



5. Frontend Hosting in S3

Frontend Code: Static website.

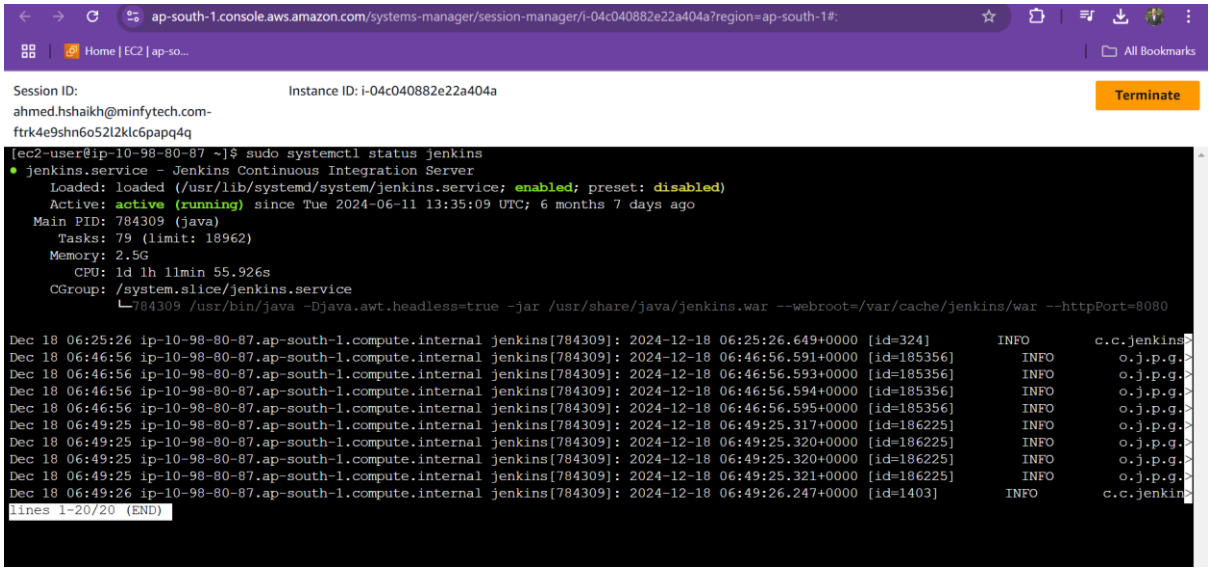


Deployment Pipeline:

Jenkins automates the build and deployment process, ensuring the latest code is always available.

6. Jenkins Setup

Jenkins is installed in our ec2 instance and it is running on port 8080.

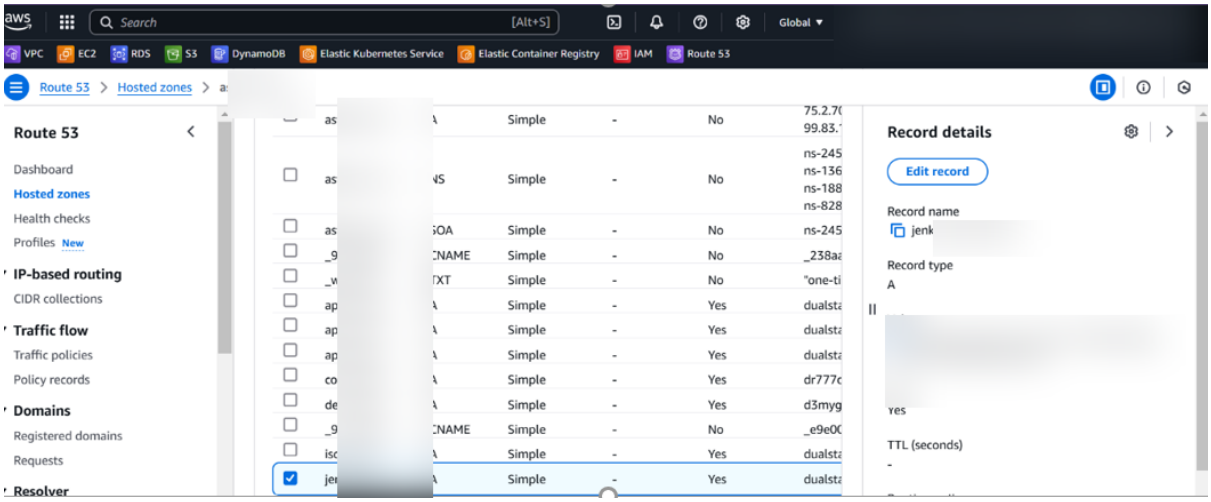


In order to access our Jenkins console we have created a subdomain for our Jenkins application.

Subdomain Name: **jenkins.astra.in**

Record Type: A Record

Value: Points to an Application Load Balancer (ALB).

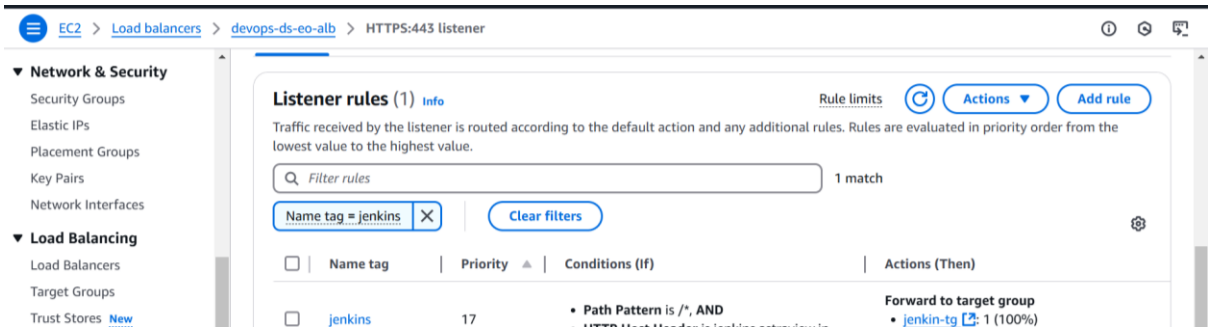


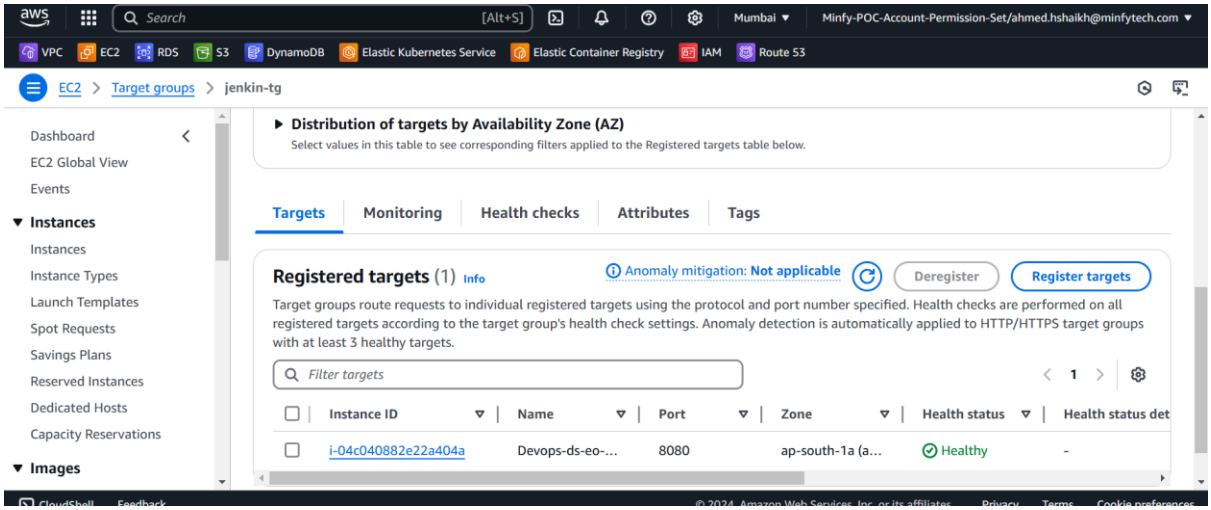
7. **ALB Configuration:**

Path Pattern: /*

Host Header: jenkins.astra.in

Target Group: Forwards traffic to EC2 instances running Jenkins on port 8080.





Here as our Jenkins is installed in our ec2 instance, so in order to access the Jenkins console we have created a subdomain named Jenkins.astra.in and routed that to point to Application Load Balancer in which we have added the target group with ec2 instance id as

our registered target with port 8080(as Jenkins runs on a port 8080. And we have added this port in the inbound rules of our security group.)

7. Pipeline Workflow

The pipeline automates the process of fetching the code, building it, and deploying the built files to the S3 bucket. Here's how it works:

Stages of the Pipeline:

a.Clone Repository

Clones the dev branch of the GitHub repository using specified credentials stored in the credentials of jenkins in the credential we have given the personal access token of our github because of which it has been integrated with our github repository.

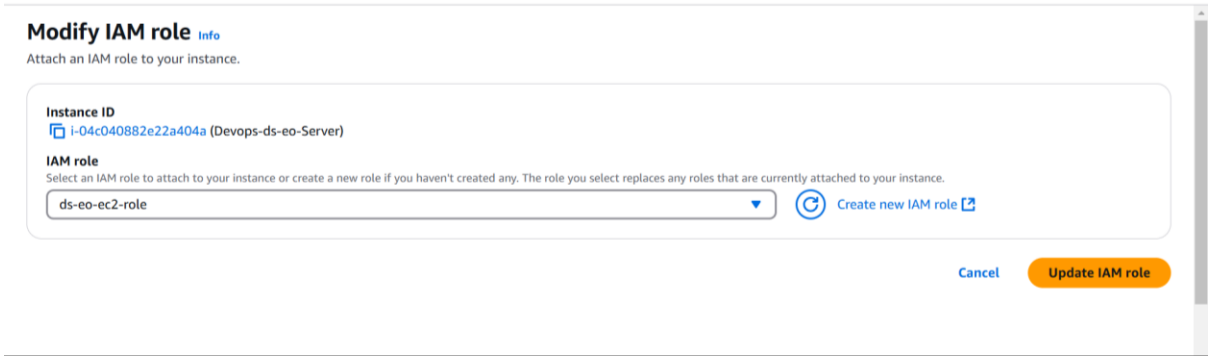
b. Fetch Environment Parameter

Here we have stored our environment variables in the parameter store as we didn't wanted it to hardcode in our code or the pipeline. So in this step it Fetches environment-specific parameters from AWS Systems Manager Parameter Store (/ds/ui/dev/env) and saves them as .env.dev.

How the Jenkins have the permission to go and fetch the details from parameter store?

Jenkins instance is running on an EC2 instance. AWS provides the concept of IAM Roles that can be attached to EC2 instances to grant permissions.

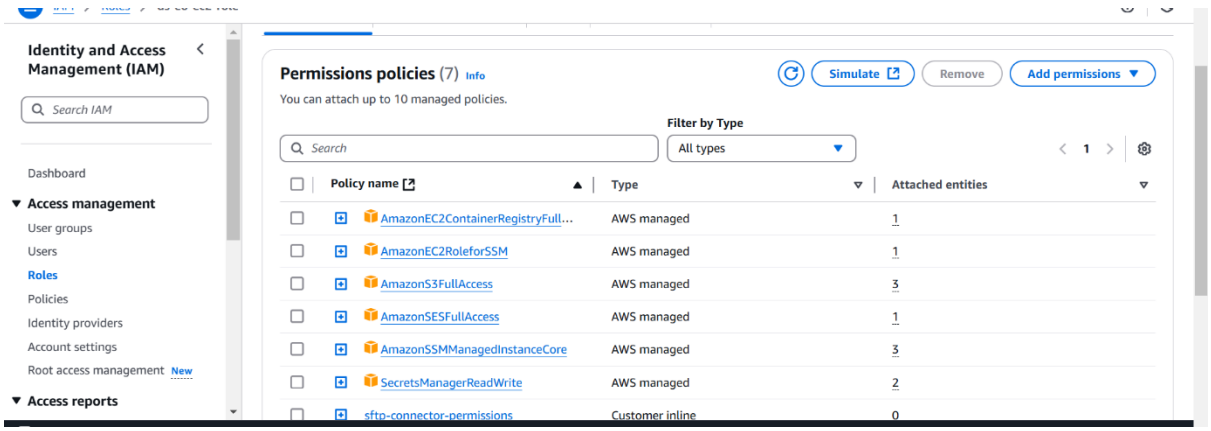
The EC2 instance hosting Jenkins is likely assigned an Instance Profile that includes an IAM role.



This IAM role contains the required permissions for AWS services, such as:

Access to SSM Parameter Store.

Access to the S3 bucket (devops-dev).



c. Install Dependencies

Installs the Node.js dependencies using npm install.

Installs additional packages, such as html-webpack-plugin.

Build the UI

Runs the npm run build-dev command to create a production-ready build of the UI. If it fails, the pipeline continues (|| true).

Remove Old UI Files from S3

Deletes all files in the S3 bucket (devops-dhri) to ensure no outdated files remain.

Upload Latest UI Files to S3

Syncs the newly built UI files from the build directory to the S3 bucket (s3://devops-dhri/).

6. End-to-End Flow

Frontend Request Flow:

User requests console.astra.in.

DNS resolves to the CloudFront distribution.

CloudFront checks:

Geo-restriction (only India-based IPs are allowed).

Serves files from the S3 bucket.

Jenkins Request Flow:

User accesses jenkins.astra.in.

DNS resolves to the ALB.

ALB forwards the request to the EC2 instance running Jenkins.

Pipeline Execution:

On every change in the GitHub repository:

Jenkins builds the frontend.

Deploys it to the S3 bucket.

S3 Files:

CloudFront fetches and caches the files from S3 for global delivery.